

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/330900993>

Evolving General Regression Neural Networks using Limited Incremental Evolution for Data-Driven Modeling of Non-linear Dynamic Systems

Conference Paper · January 2018

DOI: 10.1109/SSCI.2018.8628909

CITATIONS

5

READS

221

4 authors, including:



Ahmad Jobran Al-mahasneh

UNSW Sydney

15 PUBLICATIONS 100 CITATIONS

[SEE PROFILE](#)



S.G. Anavatti

Australian Defence Force Academy

250 PUBLICATIONS 2,473 CITATIONS

[SEE PROFILE](#)



Matt Garratt

UNSW Sydney

211 PUBLICATIONS 2,128 CITATIONS

[SEE PROFILE](#)

Some of the authors of this publication are also working on these related projects:



Special Issue on "Advanced Soft Computing for Prognostic Health Management" [View project](#)



Identifying differences in brain activity and an accurate detection of ADHD using rs-fMRI [View project](#)

Evolving General Regression Neural Networks using Limited Incremental Evolution for Data-Driven Modeling of Non-linear Dynamic Systems

1st Ahmad Jobran Al-Mahasneh

*School of Engineering and Information Technology
The University of New South Wales Canberra
Canberra, ACT 2612, Australia
ahmad.al-mahasneh@student.adfa.edu.au*

3rd Matthew A. Garratt

*School of Engineering and Information Technology
The University of New South Wales Canberra
Canberra, ACT 2612, Australia
M.Garratt@adfa.edu.au*

2nd Sreenatha G. Anavatti

*School of Engineering and Information Technology
The University of New South Wales Canberra
Canberra, ACT 2612, Australia
a.sreenatha@adfa.edu.au*

4th Mahardhika Pratama

*School of Computer Science and Engineering
Nanyang Technological University
Singapore, 639798
mpratama@ntu.edu.sg*

Abstract—In this paper, an evolutionary general regression neural network is developed based on limited incremental evolution and distance-based pruning to online model dynamic systems. Also, a variance-based method is suggested to adapt the smoothing parameter in GRNN for online applications. The proposed model is compared with different types of dynamic neural networks. A nonlinear benchmarking dynamic discrete system with white Gaussian noise is used in the comparison. The results are compared in terms of the prediction error and the time required for adaption and the comparison results show that the proposed model is more accurate and quicker than any another counterpart.

Index Terms—General Regression Neural Network, online modeling, data-driven modeling, evolutionary neural networks.

I. INTRODUCTION

Modeling non-linear dynamic systems is a demanding task which requires a deep knowledge of the physics of a given plant/process. One of the proposed solutions to model these systems is to use Artificial Intelligence (AI) based techniques such as Fuzzy Logic (FL), Evolutionary Algorithms (EA) and Artificial Neural Networks (ANN). These AI-based methods do not require any explicit knowledge of the plants/processes to model them. Data-driven modeling is merely based on the observed data. To model a given plant/process, one only needs input/output data. An FL or ANN model can be trained using these data to produce a smart prediction of the dynamics of that plant/process. The only drawback of the data-driven modeling approach is that it models plants/processes as a black-box which is not useful for those who want to understand the dynamics of these plants/processes extensively.

Since ANNs have universal approximation abilities, they can be used to approximate any non-linear continuous function

under certain conditions [1], [2]. Also, ANNs can learn the patterns in the data. Because of their abilities, ANNs can be applied to a wide range of problems including regression, prediction, pattern recognition and classification.

There are many types of ANNs, such as the Multi-Layer Perceptron (MLP), Recurrent Neural Networks (RNN) and Radial Basis Functions Network (RBFNN). In this paper, General Regression Neural Networks (GRNN) are considered. GRNN is a single-pass learning network which uses Gaussian activation functions in the hidden layer. GRNNs consist of input, hidden, summation and division layers as shown in Fig 1.

The output of GRNN can be written as:

$$\hat{Y} = \frac{\sum_{i=1}^N W_{h_o} e^{(-D/2\sigma)^2}}{\sum_{i=1}^N e^{(-D/2\sigma)^2}} \quad (1)$$

where W_{h_o} is the hidden-output layer weight, D is the Euclidean distance between the input x and the input weight W_{i_h} which can be written as $D = (x - W_{i_h})^T (x - W_{i_h})$ and σ is the smoothing parameter of the Gaussian hidden layer activation function.

The above equation can be written as an algorithm in Algorithm 1:

GRNN can be used in approximation, prediction, modeling and control tasks. GRNN have been used to model dynamic plants [3], nonlinear system identification [4], batch processes modeling and monitoring [5]. Also, in fault detection and diagnosis in air-handling unit [6], control of microgrid hybrid power systems [7], dead-zone estimation and compensation in control of traveling wave ultrasonic motor [8].

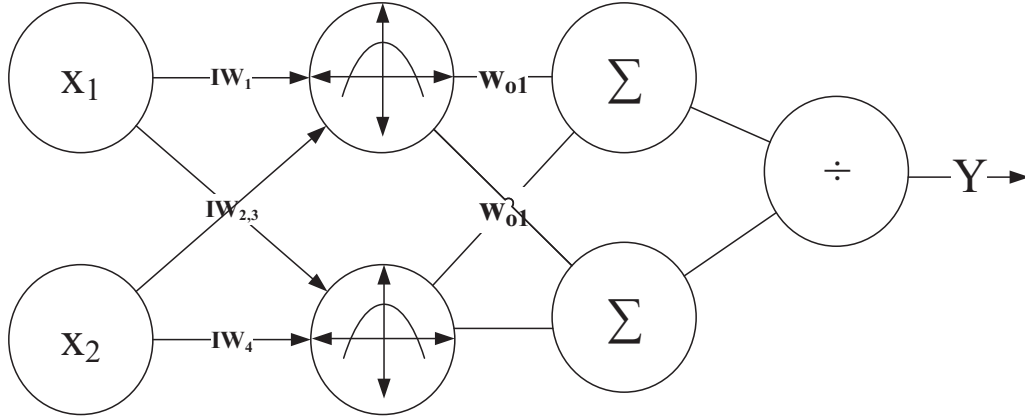


Fig. 1: GRNN structure

Algorithm 1 GRNN output

```

1: procedure GRNN OUTPUT( $\hat{Y}$ )
2:   for  $x=x_{first}$  to  $x_{final}$  do
3:     Find the distance  $D$  between  $x$  and  $IW$ 
4:     Apply the activation function  $f = e^{(-D/2\sigma)^2}$ 
5:   end for
6:   for  $x=x_{first}$  to  $x_{final}$  do
7:     Multiply every  $f$  in its output weight to find  $\bar{f}$ 
8:   end for
9:   Find  $\hat{Y} = \sum_{i=1}^N \bar{f}_i / \sum_{i=1}^N f_i$ 
10: end procedure

```

Quick and accurate learning are significant advantages of GRNN. GRNN converge to the underlying regression surface even with a few data samples [9]. A GRNN trains itself by memorizing all the distinct training patterns, so the only free parameter in GRNN is the smoothing parameter σ . Although this training procedure is quick and straightforward, it is not efficient especially if the input data have a high degree of similarity. The main contributions in this paper are:

- 1) Proposing a variance-based method to adapt the GRNN smoothing parameter σ for online modeling of dynamic systems.
- 2) Proposing Limited Incremental Evolution of GRNN (LIEV-GRNN), which grows GRNN to a certain limit and at the same time eliminates the most redundant connections in GRNN distance-based pruning methods. These structure pruning methods include input-input weights distance and target-output weights distance. The focus of evolution is to improve GRNN for modeling non-linear dynamic plants in the sequential mode.
- 3) A discrete-time non-linear plant is used to test the LIEV-GRNN and the results are compared with different types of dynamic neural networks.

The LIEV-GRNN algorithm is efficient when it adds nodes to GRNN, so it only adds nodes if the prediction error increased

more than a certain limit not like other methods [10] which will keep adding nodes until a certain limit without any restriction. LIEV-GRNN is inspired by our previous research on GRNN growing for system identification of a flapping wing micro aerial vehicle [11]. Some kernel-based pruning approaches suggested removing the oldest active example [12], [13], this method could be applied if GRNN is used in the batch-mode but in the online training mode, removing the oldest active sample will not produce the required results. GRNN output is a normalized weighted sum of the hidden kernels, so theoretically every kernel in the hidden layer contributes to the output by a certain value which means removing any kernel will affect the output.

The rest of the paper is organized as follows: Section II discusses the research tools and methodology, Section III provides some results and discussions and Section IV draw conclusions and suggest some future research ideas.

II. TOOLS AND METHODS

In this section, the details of simulation and the research methodology are discussed. The simulation is carried out in MATLAB environment for 50 seconds with a sample rate of 0.05 seconds. HP Z230 workstation is used with Intel i7 quad-core processor and 16 GB RAM. The PC is running Windows 7 operating system. GRNN initially contains only one training sample $(x,y)=(1,1)$, and a smoothing parameter (σ) of 0.1. GRNN input is the plant input $u(k)$ and the target is the current plant output $y(k)$ and GRNN is used to predict the next output $y(k+1)$ as shown in Fig. 2.

The smoothing parameter in GRNN is a major parameter which controls the smoothness and the accuracy of GRNN output. To demonstrate the importance of this parameter, GRNN is used to fit the cosine function and different values of σ are used as shown in Fig. 3. One can observe that selecting the correct σ will result in better performance.

The known methods for finding the smoothing parameter are holdout and wiggles methods. The holdout is a numerical search method based on removing one training sample and

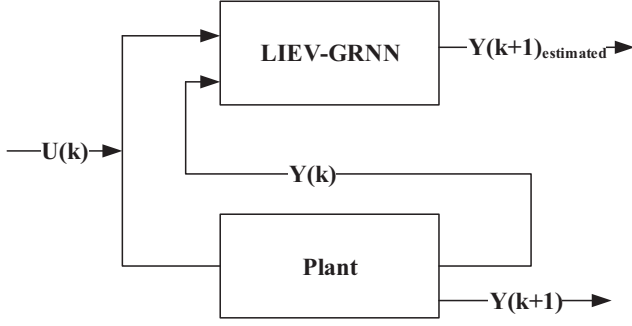


Fig. 2: LIEV-GRNN in action

training the GRNN with the remaining ones then finding GRNN output for the removed sample and repeating this procedure for different values of σ then selecting σ which has the lowest error. The main issue with this method is it is very slow since it needs to train the network many times and it needs the network output as well, so it is not suitable for online applications.

The wiggle method is another numerical approach which adjusts the smoothing parameter based on a certain limit of the number of the inflection points which can be found based on the second derivative numerically. In the wiggle method σ is initially chosen minimal value, then σ is increased gradually until the number of inflection points is less than the maximum limit. Eventually, σ which produces smooth curve will be found. This method also needs a long computation time and it is also not suitable for online applications. That is why we developed the variance method to solve the issues in the holdout and wiggle methods which make them unsuitable for online applications. So we propose a variance-based method to adapt GRNN smoothing parameter online.

After initialization, the limited incremental evolution starts if the LIEV-GRNN prediction error is more than 0.001. The maximum allowed size of the hidden layer in LIEV-GRNN is 10 neurons. The smoothing parameter of GRNN (σ) is adapted according to the variance of the input data as follows:

$$\sigma_{GRNN} = \frac{\sum_{i=1}^N (u(i) - \mu)^2}{N} \quad (2)$$

where $u(i)$ is the plant input, μ is the mean of the plant input data and N is the total number of the input data. It should be also mentioned that the values of $u(i)$ should be normalized to keep the smoothing parameter values less than one since after one the output will be smooth but the fit is not accurate.

In the LIEV-GRNN, initially GRNN needs small values of σ which means more kernels are used find the output accurately, but after learning, the value of σ can be increased, so fewer kernels should be used to find the output. In the case of LIEV-GRNN, since it is incremental, its size increases, so initially small values of σ are required, but after more kernels are added in incremental evolution, the values of σ can be increased. Also, the smoothing parameter should be

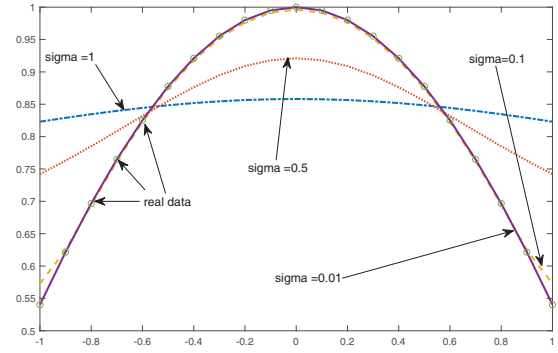


Fig. 3: Effect of smoothing parameter on GRNN fitting

smaller than the average distance between the inputs to fit the data firmly [14]. The variance is suitable to adapt the GRNN smoothing parameter because it measures the distance between the data and the mean. The changes of σ values through the simulation is shown in Fig. 4.

A. Incremental Evolution

Incremental Evolution starts with a minimal structure GRNN then it will be grown based on specific criteria. The incremental approach is suitable for GRNN when it is used in the modeling and control of dynamics systems since GRNN structure should be modified when a new input data comes. In the suggested method GRNN evolution algorithm will only be active if the prediction error increased over a user-specified limit.

1) *Limited Incremental Evolution of GRNN*: The proposed evolutionary approach is 'limited' because it limits the maximum possible size of GRNN. On the other hand, the unlimited incremental evolution will keep growing GRNN without an upper size limit. In this research, limited incremental evolution is adopted to keep GRNN as compact as possible which will save computation time and memory. Algorithm 2 explains the Limited Incremental Evolution GRNN.

Algorithm 2 Limited Incremental Evolution GRNN (LIEV-GRNN)

```

1: procedure LIEV-GRNN(GRNN)
2:   for  $t=0$  to  $t_{final}$  do
3:     Find the Mean Square Error  $e(t)$ 
4:     if  $e(t) > \max(e(t))$  then
5:       if  $(\text{size}(\text{GRNN})_i \text{Max}(\text{size}))$  then
6:         Add connection
7:       end if
8:       Evolve the current structure (pruning)
9:     end if
10:  end for
11: end procedure

```

TABLE I: LIEV-GRNN results

Type	RMSE	Time per iteration (sec)
Input distance pruning	0.105	8.906e-04
Output distance pruning	0.138	9.548e-04

B. Pruning and Creating Connections and Neurons in GRNN

Besides incremental evolution, pruning is also introduced which will reduce GRNN prediction error by removing the least significant connections.

Removing and addition of connections and neurons in GRNN is based on the Euclidean distance between the input and the input weights or based on the Euclidean distance between the target and the output weights as expressed in (3) and (4). The connection which have the highest distance will be pruned. In GRNN when removing a neuron its connection should also be removed, so the neuron, input-hidden and hidden-output layer connections will be removed.

$$f(IW) = (x - IW)^T(x - IW) \quad (3)$$

$$f(OW) = (t - OW)^T(t - OW) \quad (4)$$

where x is the input, IW is the input weight matrix, t is the target and OW is the output weights matrix.

III. RESULTS AND DISCUSSION

A benchmarking discrete non-linear plant from [3] is used for simulation. The plant can be written as:

$$y(k+1) = \frac{y(k)y(k-1)(2.5 + y(k-1))}{(1 + y(k)^2 + y(k-1)^2)} + \frac{2e^{-u(k)} - 1}{e^{-u(k)} + 1} + \epsilon(k) \quad (5)$$

where $\epsilon(k)$ is white Gaussian noise with zero mean as shown in Fig. 5.

The input to the system $u(k)$ is given by:

$$u(k) = 0.2\sin(0.5t) + 0.5\sin(0.3t) \quad (6)$$

The results of LIEV-GRNN identification in both the input distance and the output distance pruning are shown in TABLE I. LIEV-GRNN is accurate and takes a short time to remove the least significant neurons.

A. LIEV-GRNN Vs. MLP and Recurrent Neural Networks

In this section, a comparison between LIEV-GRNN and adaptive dynamic MLP network is conducted. MLP is adapted if the prediction error MSE is more than 0.001. The adaptation is a gradient-descent method and the hidden layer contains ten neurons which is the maximum size limit for LIEV-GRNN as shown in Fig. 6. This limit is to avoid overfitting. The results show that LIEV-GRNN is more accurate than the dynamic MLP and it takes significantly shorter time than the MLP.

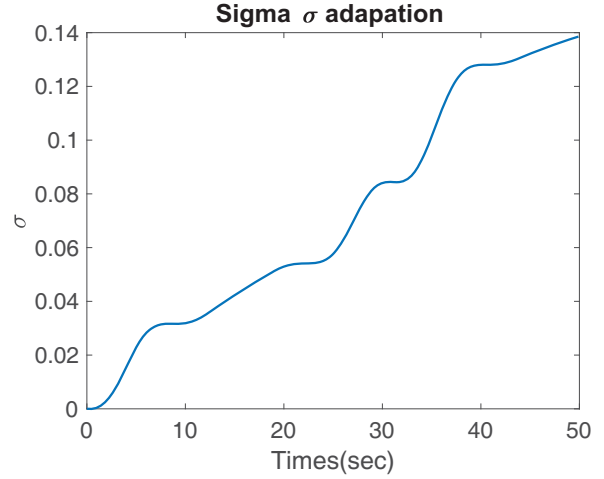
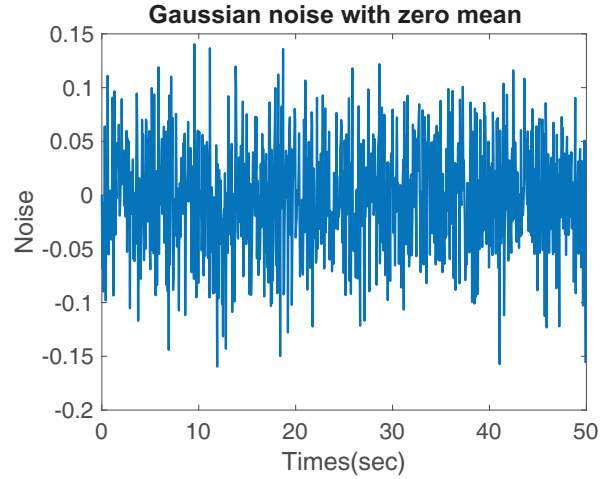
Fig. 4: Smoothing parameter σ adaptation

Fig. 5: Gaussian noise with zero mean

Also, the layer recurrent neural network as shown in Fig 7 and the Nonlinear AutoRegressive Neural Network with External input (NARX) as shown in Fig 8 are used to identify the same plant as in (6). RANN have feedback either from the hidden layer output such as The layer recurrent or the output such as NARX. RANNs are used widely in time series prediction. In this experiment, RANN are adaptive, so once the prediction error became more than 0.001 their weights are adapted. The size of the hidden layer in both of the networks is ten neurons. The training function of MLP, RANNs is Levenberg-Marquardt.

The results show that the RANN and the dynamic MLP prediction errors are around 0.4 and they need approximately 0.02 seconds for each adaptation.

Based on the simulation results, it can be confirmed that LIEV-GRNN results (see Fig. 9) has better accuracy than the dynamic MLP (see Fig. 10), the layer recurrent (see Fig. 11) and the NARX Fig. 12. The difference between the estimated

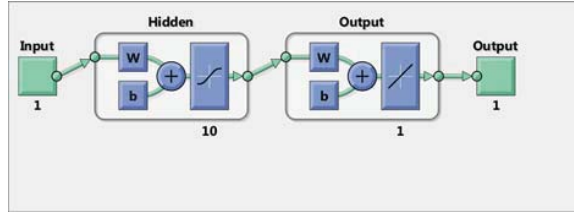


Fig. 6: MATLAB view of MLP

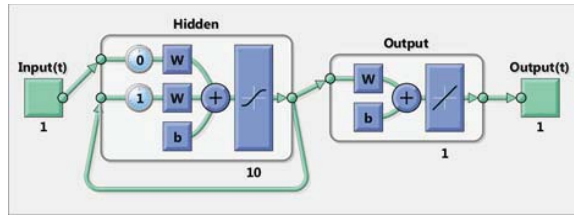


Fig. 7: MATLAB view of Layer recurrent NN

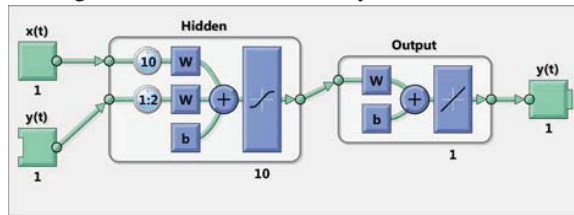


Fig. 8: MATLAB view of NARX

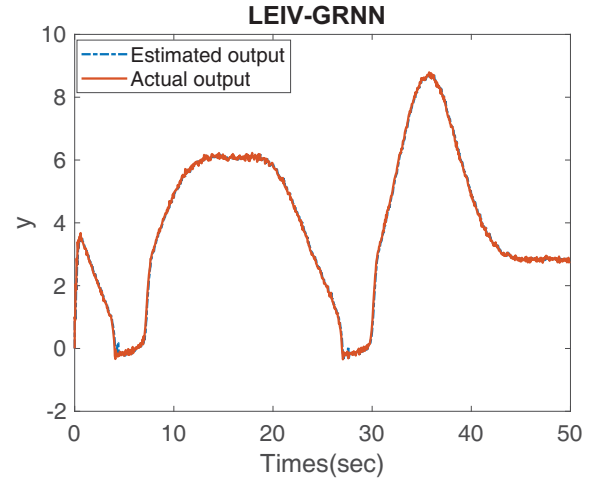


Fig. 9: LIEV-GRNN

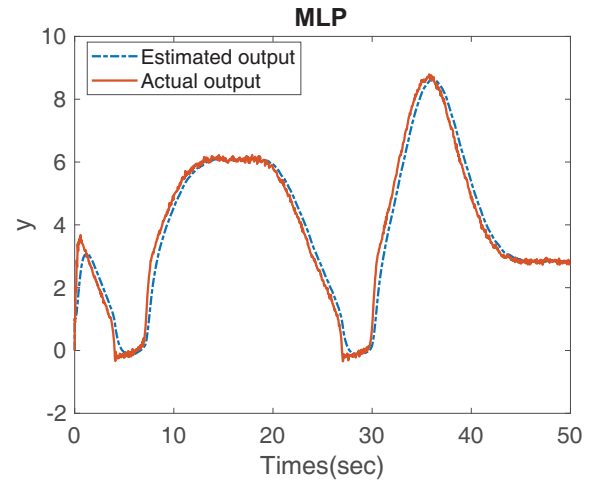


Fig. 10: MLP output

and the actual outputs for all the used models is shown in Fig. 13. Also, the comparison between the different models in the Root Mean Square Error (RMSE) and the time required for each adaptation is shown in TABLE II. LIEV-GRNN has the lowest error and the quickest adaptation time.

TABLE II: LIEV-GRNN Vs dynamic ANNs

Type	RMSE	Time per iteration (sec)
LIEV-GRNN	0.105	8.906e-04
Dynamic MLP	0.425	0.0212
Layer recurrent NN	0.397	0.0200
NARX NN	0.414	0.0224

IV. CONCLUSIONS AND RECOMMENDATIONS

LIEV-GRNN is a new generation of GRNN which provides a quick and accurate solution to model non-linear dynamics systems even in the presence of noise.

Incremental evolution and pruning techniques provide a solution to optimize the size of GRNN without losing the prediction accuracy.

The variance-based smoothing factor adaptation helps to improve GRNN performance and proves that the smoothing factor should not be fixed when using GRNN in online applications since adapting it will improve the accuracy and smoothness of GRNN fitting.

The distance between the input weights and the inputs and the distance between the output weights and the targets are used to prune the least significant neurons in GRNN.

More pruning strategies should be developed in the future research.

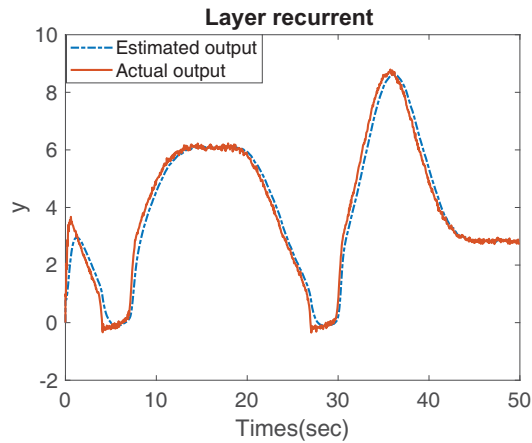


Fig. 11: Layer recurrent output

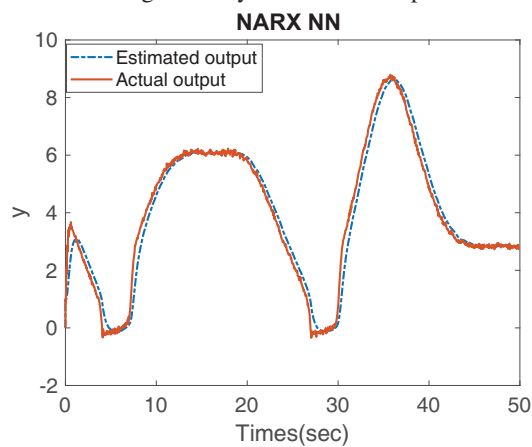


Fig. 12: NARX output

The next generation of the LIEV-GRNN will be directed to the control of the dynamic systems.

REFERENCES

- [1] J. Park and I. W. Sandberg, "Universal approximation using radial-basis-function networks," *Neural computation*, vol. 3, no. 2, pp. 246–257, 1991.
- [2] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural networks*, vol. 2, no. 5, pp. 359–366, 1989.
- [3] T. Seng, M. Khalid, and R. Yusof, "Adaptive general regression neural network for modelling of dynamic plants," *Proceedings of the Institution of Mechanical Engineers, Part I: Journal of Systems and Control Engineering*, vol. 213, no. 4, pp. 275–287, 1999.
- [4] O. Nelles, *Nonlinear system identification: from classical approaches to neural networks and fuzzy models*. Springer Science & Business Media, 2013.
- [5] S. G. Kulkarni, A. K. Chaudhary, S. Nandi, S. S. Tambe, and B. D. Kulkarni, "Modeling and monitoring of batch processes using principal component analysis assisted generalized regression neural networks," *Biochemical Engineering Journal*, vol. 18, no. 3, pp. 193–210, 2004.
- [6] W.-Y. Lee, J. M. House, and N.-H. Kyong, "Subsystem level fault diagnosis of a building's air-handling unit using general regression neural networks," *Applied Energy*, vol. 77, no. 2, pp. 153–170, 2004.
- [7] T.-C. Ou and C.-M. Hong, "Dynamic operation and control of microgrid hybrid power systems," *Energy*, vol. 66, pp. 314–323, 2014.
- [8] T.-C. Chen and C.-H. Yu, "Motion control with deadzone estimation and compensation using grnn for twusm drive system," *Expert Systems with Applications*, vol. 36, no. 8, pp. 10931–10941, 2009.
- [9] D. F. Specht, "A general regression neural network," *IEEE transactions on neural networks*, vol. 2, no. 6, pp. 568–576, 1991.
- [10] K. Yamauchi, "Pruning with replacement and automatic distance metric detection in limited general regression neural networks," in *Neural Networks (IJCNN), The 2011 International Joint Conference on*. IEEE, 2011, pp. 899–906.
- [11] A. J. Al-Mahasneh, S. G. Anavatti, and M. A. Garratt, "Altitude identification and intelligent control of a flapping wing micro aerial vehicle using modified generalized regression neural networks," in *Computational Intelligence (SSCI), 2017 IEEE Symposium Series on*. IEEE, 2017, pp. 1–6.
- [12] O. Dekel, S. Shalev-Shwartz, and Y. Singer, "The forgetron: A kernel-based perceptron on a fixed budget," in *Advances in neural information processing systems*, 2006, pp. 259–266.
- [13] Y. Kondo and K. Yamauchi, "A dynamic pruning strategy for incremental learning on a budget," in *International Conference on Neural Information Processing*. Springer, 2014, pp. 295–303.
- [14] C. Li, A. C. Bovik, and X. Wu, "Blind image quality assessment using a general regression neural network," *IEEE Transactions on Neural Networks*, vol. 22, no. 5, pp. 793–799, 2011.

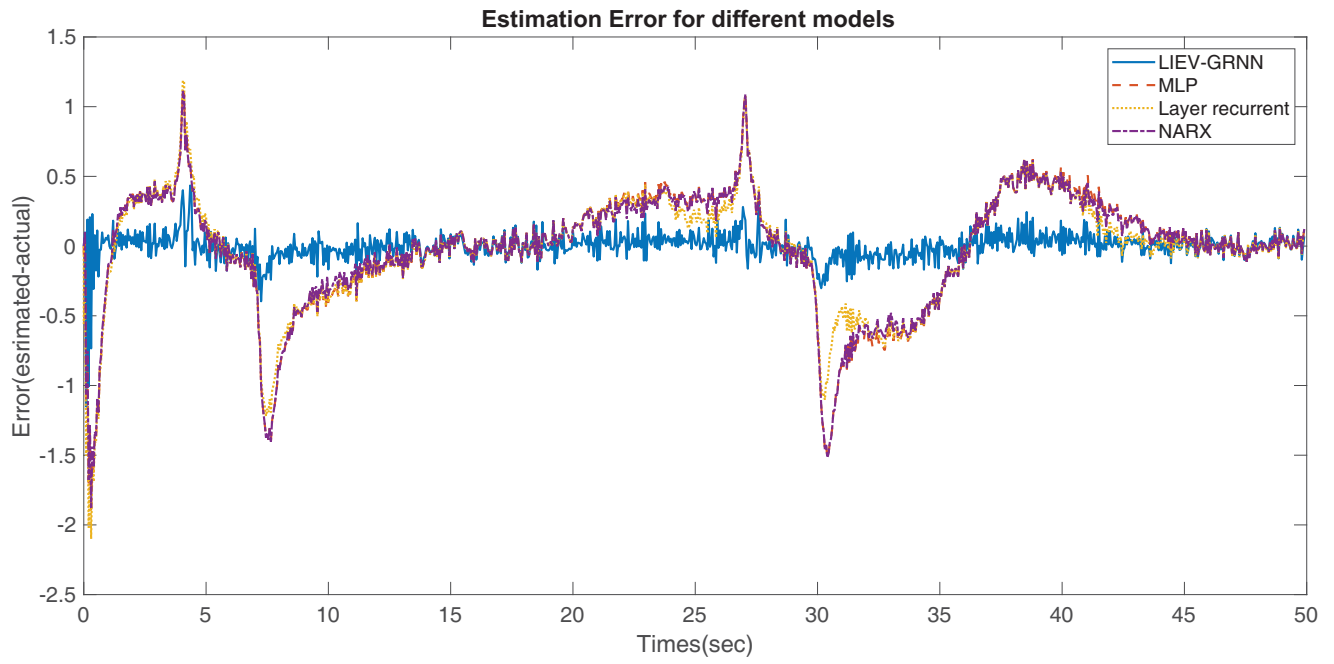


Fig. 13: Comparison between the error in the different models